IS834 - Data Analysis Cheatsheet

Table of contents

This cheatsheet provides quick code examples to help you analyze data using Python's pandas and related tools.

0. Getting Started

Getting Help

```
# define a variable
x = 2

# get help
?x
help(x)
help("&")

# or on a method
?pd.read_csv
```

Installing Python Libraries

```
# in a notebook (.ipynb file)
! pip install pandas
```

In the terminal

```
pip install pandas
```

1. Loading Datasets into Pandas

```
import pandas as pd

# Load CSV file
df = pd.read_csv("data.csv")

# Load Excel file
df_excel = pd.read_excel("data.xlsx", sheet_name="Sheet1")
```

> **!** Important
>
> Above assumes that the CSV and Excel files are located in your **current working directory**. On our machine, or in Positron, you should able able to right-click and copy a (relative) path to help you, if Intellisense does not pop up for you.

2. Getting Information on the Dataset

```
# Basic overview
df.info()

# the shape (rows, columns)
df.shape

# Summary statistics for numeric columns
df.describe()

# Sumamry stastitics for all columns
df.describe(include='all')

# View first/last rows
df.head()  # first 5 rows
df.tail()  # last 5 rows

# this can also be for a column in a pandas DataFrame
df['column1'].describe()

# if the column name doesn't have spaces
df.column1.describe()

# how many unique values
df.column1.nunique()

# only return the unique values
df['column1'].unique()
```

4. Filtering Rows and Columns

```
# take a sample of rows
df.sample()

# Select specific columns
df_filtered = df[["column1", "column2"]]

# Select certain rows
row_filter = df[df.col_a > 5]

# Filter rows using conditions (using .loc and .iloc is more verbose but perhaps easier to read)
df_filtered = df.loc[df["column"] > 10, :]

# Filter rows and select specific columns
df_filtered = df.loc[df["column"] == "value", ["column1", "column2"]]

# multiple filter conditions
df_filtered = df.loc[(df["column1"] == "value") & (df["column2"] != "value2")), ["column1", "column2"], :]

# drop duplicates
df2 = df.drop_duplicates()

# drop rows that have missing values
df2_no_missing = df.dropna()

# we can specify column(s) to inspect in order to only evaluate missing against a subset of data
df2_no_missing = df.dropna(subset=['column1', 'column2'])
```

4. Column Management

```
# Create a new column from existing columns
df["new_column"] = df["column1"] + df["column2"]
```

```python
# Apply a transformation
df["column_squared"] = df["column"] * df["column"]

# apply a function - needs to be applied to proper datatypes
def plus1(x):
  return x+1
df['plus_one'] = df['numeric_column'].apply(plus1)

# Apply a lambda (temporary) function
df["column_squared"] = df["column"].apply(lambda x: x * 2)

# rename columns
rename_mapper = {'old_column_name': 'new_column_name'}
df = df.rename(columns = rename_mapper)

# delete/drop columns
df = df.drop(columns = ['col1'])

# keep just numeric cols
df_numeric = df.select_dtypes("number")

# change the column type
df['col1'] = df['col1'].astype(int)

# lower level force numeric if there are missing values
df['col1'] = pd.to_numeric(df.col1, errors='coerce')

# replace values in a specific column
df['col1'] = df['col1'].replace({'old_value':'new_value'})

# replace all values in the dataframe
df = df.replace({'old_value':'new_value'})
```

5. Working with Missing Data

```
# Check for missing values
df.isna().sum()

# Fill missing values
df_filled = df.fillna(value={"column": 0})

# Drop rows with missing values - specific columns evaluated
df_dropped = df.dropna(subset=["column1", "column2"])

# getting help
df.dropna?
```

6. String and Datetime Operations

```
# String operations - note the .str accessor
df["column"] = df["column"].str.lower()

# Convert to datetime, including if formats are of different types!
df["date_column"] = pd.to_datetime(df["date_column"], format="mixed")

# Extract year from datetime dtype variable - note the .dt accessor
df["year"] = df["date_column"].dt.year
```

7. Grouping and Pivoting

```

# univariate counts with value_counts
df['column'].value_counts()

# include missing in the output
```

```python
df['column'].value_counts(dropna=False)

# Group by and aggregate
grouped = df.groupby("category_column", as_index=False)["value_column"].sum()

# multiple columns
grouped = df.groupby(['col1', 'col2'], as_index=False)["value_column"].mean()

# use of .agg
grouped = df.groupby("category_column", as_index=False).agg({'column_1': ['mean', 'sum', 'max'], 'another_column': 'min'}).reset_

# we can also use named values with .agg
grouped = df.groupby("category_column", as_index=False).agg(
    mean_col1 = ('column_1', 'mean'),
    sum_col2 = ('another_column', 'sum')
)

# or dictionaries to help with the aggregation
grouped = df.groupby("category_column", as_index=False).agg(
    {
        'column_1': 'mean',
        'another_column': 'min'
    }
)

# Pivot table allow us to put data in the rows and columns
pivot = df.pivot_table(values="value_column", index="row_column_name(s)", columns="column(s)_name", aggfunc="mean")

# Cross tabulation - simple count table with two rows and columns
# this is a frequency (count) table but we can control aggregation of a variable!
crosstab = pd.crosstab(df["column1"], df["column2"], margins=True)
```

8. Plotting Basic Charts with Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

# count plot
sns.countplot(data=df, x="col1")

# grouped bar plot
sns.countplot(data=df, x="cut", hue="color")

# Scatter plot
sns.scatterplot(data=df, x="column1", y="column2")

# Bar plot
sns.barplot(data=df, x="category_column", y="value_column")

# heatmap with different color map pallette
sns.heatmap(df_numeric.corr(), cmap="Blues")

# seaborn has line plots
sns.lineplot(data=df, x="cut", y="price")

# histogram
sns.histplot(data=df, x="price")

# boxplot
sns.boxplot(data=df, x="cut", y="price")
```

You can configure plots with `.plt`.

```
# reference line
plt.axhline(df.col.mean(), color="red", linestyle="--")

plt.title("My First Plot")
plt.xlabel("My Column on the X-Axis")
```

```
plt.ylabel("Y-Axis Column")

plt.show()  #render the plot
```

9. Basic Statistical Analysis

```
# Mean, median, mode
mean_value = df["column"].mean()
median_value = df["column"].median()
mode_value = df["column"].mode()[0]

# Correlation matrix -
correlation = df_numeric.corr()
```

If you want to perform statistical tests, you can use the `Pingouin` package.

```
import pingouin as pg

# t-test
pg.ttest(x, y)

# anova
aov = pg.anova(data=df, dv='numeric_column', between='group_column', detailed=True)

# chi-square test
expected, observed, stats = pg.chi2_independence(data=diamonds, x="cut", y="clarity")
stats

# linear regression
reg = pg.linear_regression(X=diamonds.carat, y=diamonds.price)

# multiple linear regression
X = df.loc[:, ['carat', 'depth']]
reg2 = pg.linear_regression(X=X, y=df.price)
```

10. Database

We used duckdb with python to demonstrate how we can use a database as the source of analysis.

To install duckdb in a notebook

```
! pip install duckdb
```

or in a terminal

```
pip install duckdb
```

Connect to an existing database.

> NOTE:
>
> If a duckdb database exists at the file location you provide, Python will connect and use the data withint he database. **If a database does not exist at the path you supply** duckdb **will create** a new database for you.

```
import duckdb

# connect to the database
db = duckdb.connect("my-database-name.duckdb")

# list the tables
db.sql("SHOW TABLES;")

# describe a table
db.sql("DESCRIBE tablename;")

# retrieve data from a table into a pandas dataframe
df = db.sql("select * from tablename").df()

# NOTE: we should use limit clauses when exploring data but this will only return 10 rows to preview
df = db.sql("select * from tablename limit 10")
```

```
# we can select specific columns
df = db.sql("select col1, col2 from tablename limit 10")

# we can filter to retrieve specific data
df = db.sql("select col1, col2 from tablename where col1 > 5 limit 10")

# we create add a new table to the database using the contents of our pandas dataframe
db.sql("create table new_table as select * from pandas_df_name")

# if we have an existing table and a dataset with the same shape (column order, columns)
db.sql("insert into existing_table select * from pandas_df")

# close the connection
df.close()
```

11. Web Data

A common library for requesting data from cloud sources and APIs is requests

```
! pip install requests # in a notebook
pip install requests # in the terminal
```

```
import requests

# get the response
resp = requests.get(url)

# we want a status code of 200
resp.status_code()

# we can get the text of the response (if we used a website that has text e.g. news site)
resp.text()

# if calling an API, likely we are getting JSON, which should be able to be parsed into a python dictionary
resp.json()
```

Pandas also has some functionality to work with web data.

```
# this will return a list of pandas dataframes where HTML tables exist on the page
tables = pd.read_html(url)

# if analyzing data from an API that returns json
resp = requests.get(url)
df = pd.json_normalize(resp.json())
```

12. Machine Learning with Scikit-Learn

```
# not an exhaustive list, but core imports
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.tree import export_text, plot_tree
from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error, mean_absolute_percentage_error
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import silhouette_score
```

Supervised ML via Decision Trees

```
# train test split, test size can be anything you want, below is 20% for testing/evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# fit a regression tree, same can be done for a classification tree (Classifier)
tree = DecisionTreeRegressor(max_depth=4, min_samples_leaf=500)
tree.fit(X_train, y_train)

# plot the tree
```

```
plt.figure(figsize=(15, 7))
plot_tree(tree, feature_names=X_train.columns, filled=True)
plt.savefig("tree.png")

# the text rules of the tree
tree_rules = export_text(tree, feature_names=X_train.columns.tolist())
print(tree_rules)

# evaluate the model on the test set
preds = tree.predict(X_test)

# regression evaluation
print(r2_score(y_test, preds))
print(mean_absolute_error(y_test, preds))
print(mean_absolute_percentage_error(y_test, preds))

# classification evaluation
print(accuracy_score(y_test, preds))
print(classification_report(y_test, preds))
```

Unsupervised ML via KMeans

```
# keep just the numeric variables
df_numeric = df.select_dtypes(include="number")

# we can scale the variables so they are all on the same units (zscore)
# notice we are doing this via fit_transform which does fit and transform in one pass
# we can apply scaler to new datasets of the same shape in the future!
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df_numeric)

# fit a kmeans cluster solution for 3 clusters
k = 3
kmeans = KMeans(n_clusters=k)
kmeans.fit(diamonds_scaled)
```

```
# return the cluster assignments for every row
kmeans.predict(df_scaled)

# use yellowbrick to evaluate a range of K
from yellowbrick.cluster import KElbowVisualizer

model = KMeans()
visualizer = KElbowVisualizer(model, k=(3,21), metric='silhouette', timings=False)
visualizer.fit(dsample_scaled)
visualizer.show()
```